

CO3 – AT1 - Debugging

J KARTHIK

192472136

CSA0911

JAVA

1. Try – Catch – Finally Basks (Debugging)

Program:

```
public class TryCatchFinallyDemo {  
    public static void main(String[] args) {  
  
        int[] numbers = {1, 2, 3};  
  
        try {  
            System.out.println(numbers[5]);  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Caught array index error");  
        }  
  
        try {  
            int value = Integer.parseInt("abc");  
        } catch (Exception e) {  
            System.out.println("Caught number format error");  
        }  
  
        String str = null;  
  
        try {  
            System.out.println(str.length());  
        } catch (NullPointerException e) {  
            System.out.println("Caught null pointer");  
        }  
    }  
}
```

```
    } finally {  
        System.out.println("Cleanup");  
    }  
    try {  
        riskyMethod();  
    } catch (java.io.IOException e) {  
        System.out.println("Caught IOException: " + e.getMessage());  
    }  
}  
  
static void riskyMethod() throws java.io.IOException {  
    throw new java.io.IOException("failure");  
}  
}
```

Output:

Caught array index error

Caught number format error

Caught null pointer

Cleanup

Caught IOException: failure

2. Producer – Consumer with BlockingQueue

Program:

```
import java.util.concurrent.*;

public class ProducerConsumerDemo {

    public static void main(String[] args) throws InterruptedException {

        BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(5);

        Thread producer = new Thread(() -> {

            for (int i = 0; i < 10; i++) {

                try {

                    queue.put(i);

                    System.out.println("Produced: " + i);

                } catch (InterruptedException e) {

                    Thread.currentThread().interrupt();

                }

            }

        });

        Thread consumer = new Thread(() -> {

            for (int i = 0; i < 10; i++) {

                try {

                    int val = queue.take();

                    System.out.println("Consumed: " + val);

                } catch (InterruptedException e) {

                    Thread.currentThread().interrupt();

                }

            }

        });

    }
```

```
        producer.start();
        consumer.start();
        producer.join();
        consumer.join();
        System.out.println("Queue size: " + queue.size());
    }
}
```

Output:

Produced: 0

Produced: 1

Produced: 2

Produced: 3

Produced: 4

Produced: 5

Consumed: 0

Consumed: 1

Produced: 6

Produced: 7

Consumed: 2

Consumed: 3

Consumed: 4

Produced: 8

Consumed: 5

Consumed: 6

Produced: 9

Consumed: 7

Consumed: 8

Consumed: 9

Queue size: 0

3. Synchronized Methods and Race Conditions

Program:

```
public class SynchronizedCounterDemo {  
    private int count = 0;  
    public synchronized void increment() {  
        count++;  
    }  
    public synchronized void decrement() {  
        count--;  
    }  
    public static void main(String[] args) throws InterruptedException {  
        SynchronizedCounterDemo demo = new SynchronizedCounterDemo();  
        Runnable task = () -> {  
            for (int i = 0; i < 1000; i++)  
                demo.increment();  
        };  
        Thread t1 = new Thread(task);  
        Thread t2 = new Thread(task);  
        t1.start();  
        t2.start();  
        t1.join();  
        t2.join();  
        System.out.println("Final count: " + demo.count);  
    }  
}
```

Output:

Final count: 2000

